[Lecture Note] Decision Structures and Boolean Logic

MGS3001 Python Programming for Business, Spring 2025

Chad (Chungil Chae)

Tue, 27 May 2025

Learning Objectives

(Gaddis, 2022).

SHORT VIDEO INTRODUCTION

https://www.youtube.com/watch?v=K5ea4-Omu-I

The if statement

A control structure is a logical design that controls the order in which a set of statements execute. So far in this book, we have used only the simplest type of control structure: **the sequence structure**.

- A sequence structure is a set of statements that execute in the order in which they appear.

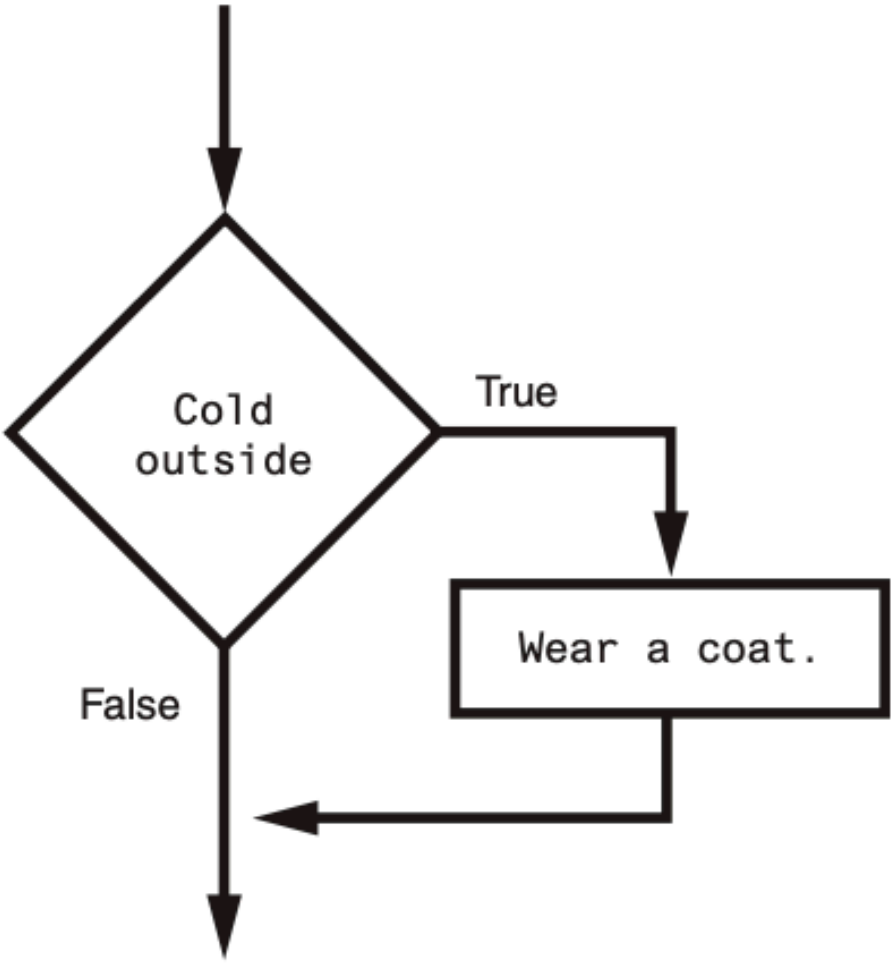
```
name = input('What is your name? ')                    ①
age = int(input('What is your age? '))                 ②
print('Here is the data you entered:')                 ③
print('Name:', name)                                   ④
print('Age:', age)                                     ⑤
```

① Get name from user input
② Get age as integer from user input
③ Display a message
④ Display name
⑤ Display age

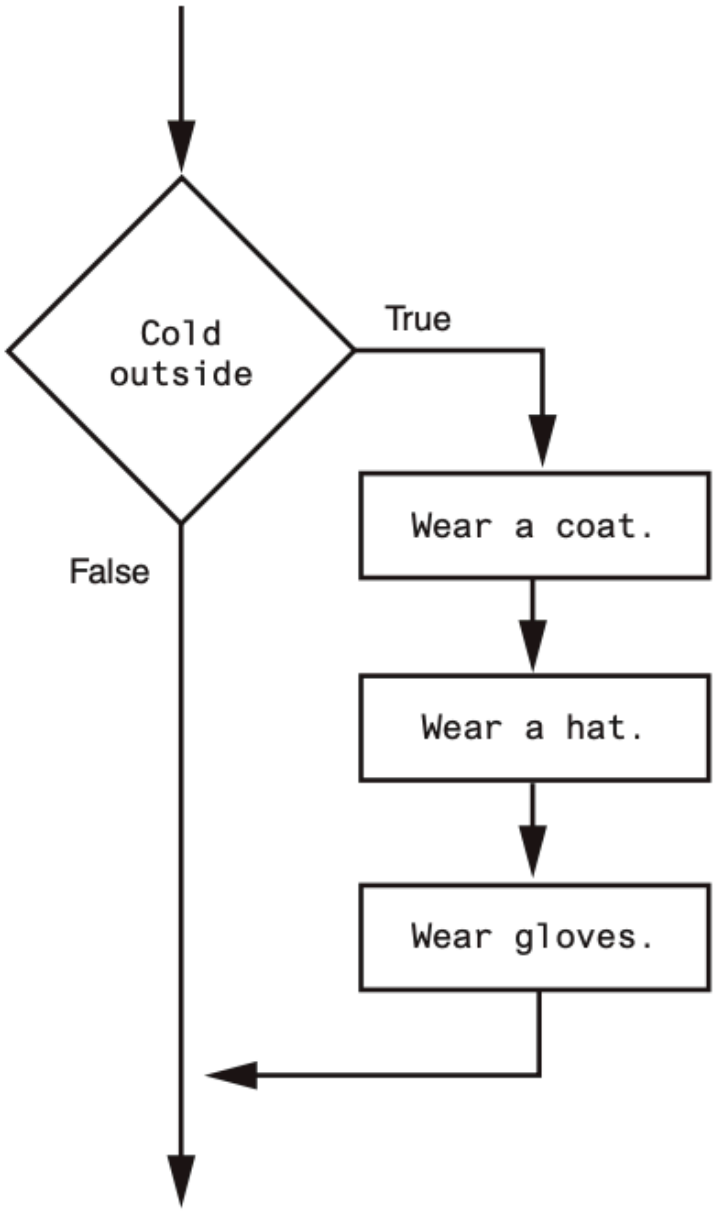
- Some problems simply cannot be solved by performing a set of ordered steps, one after the other.
  - e.g. A pay calculating program that determines whether an employee has worked overtime. If the employee has worked more than 40 hours, he or she gets paid extra for all the hours over 40. Otherwise, the overtime calculation should be skipped.

Programs like this require a different type of control structure: one that can execute a set of statements only under certain circumstances. This can be accomplished with a **decision structure.** (Decision structures are also known as selection structures.)

- In a decision structure's simplest form, a specific action is performed only if a certain condition exists.
- If the condition does not exist, the action is not performed. The flowchart shows how the logic of an everyday decision can be diagrammed as a decision structure.
- The diamond symbol represents a true/false condition.
  - If the condition is true, we follow one path, which leads to an action being performed.
  - If the condition is false, we follow another path, which skips the action.



33



- In the flowchart the diamond symbol indicates some condition that must be tested.
  - In this case, we are determining whether the condition Cold outside is true or false.
    - If this condition is true, the action Wear a coat is performed.
    - If the condition is false, the action is skipped.

  The action is conditionally executed because it is performed **only when a certain condition is true**.

- Programmers call the type of **decision structure** shownas a single alternative decision structure.

- This is because it provides only one alternative path of execution.
  - If the condition in the diamond symbol is true, we take the alternative path.
  - Otherwise, we exit the structure.

if statement to write a single alternative decision structure. Here is the general format of the if statement:

```
if condition:                                                    ①
  statement                                                      ②
  statement
  etc
```

① The first line as the if clause.
② Sequence structure

- The if clause begins with the word **if**, followed by a condition, which is an expression that will be evaluated as either true or false.
- **A colon (:)** appears after the condition.
- Beginning at the next line is a block of statements.
  - A block is simply a set of statements that belong together as a group.
- Notice in the general format that all of the statements in the block are indented.
  - This indentation is required because the Python interpreter uses it to tell where the block begins and ends.
- When the if statement executes, the condition is tested.
  - If the condition is true, the statements that appear in the block following the if clause are executed.
  - If the condition is false, the statements in the block are skipped.

Boolean Expressions and Relational Operators

- The if statement tests an expression to determine whether it is true or false.
- The expressions that are tested by the if statement are called Boolean expressions, named in honor of the English mathematician George Boole.
- In the 1800s, Boole invented a system of mathematics in which the abstract concepts of true and false can be used in computations.
- Typically, the Boolean expression that is tested by an if statement is formed with a relational operator. A relational operator determines whether a specific relationship exists between two values.
- For example, the greater than operator (>) determines whether one value is greater than another. The equal to operator (==) determines whether two values are equal.

Relational operators

Operator	Meaning
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to
!=	Not equal to

71 Boolean expressions using relational operators

Operator	Meaning
x > y	Is x Greater than y?
x < y	Is x Less than y?
x >= y	Is x Greater than or equal to y?
x <= y	Is x Less than or equal to y?
x == y	Is x Equal to y?
x != y	Is x Not equal to y?

72 ---

```
x = 1
y = 0
x > y
```

```
x = 1
y = 0
y > x
```

```
x = 1
y = 0
z = 1
x >= y
x <= z
x <= y
```

```
x = 1
y = 0
z = 1
x >= y
x <= z
x <= y
```

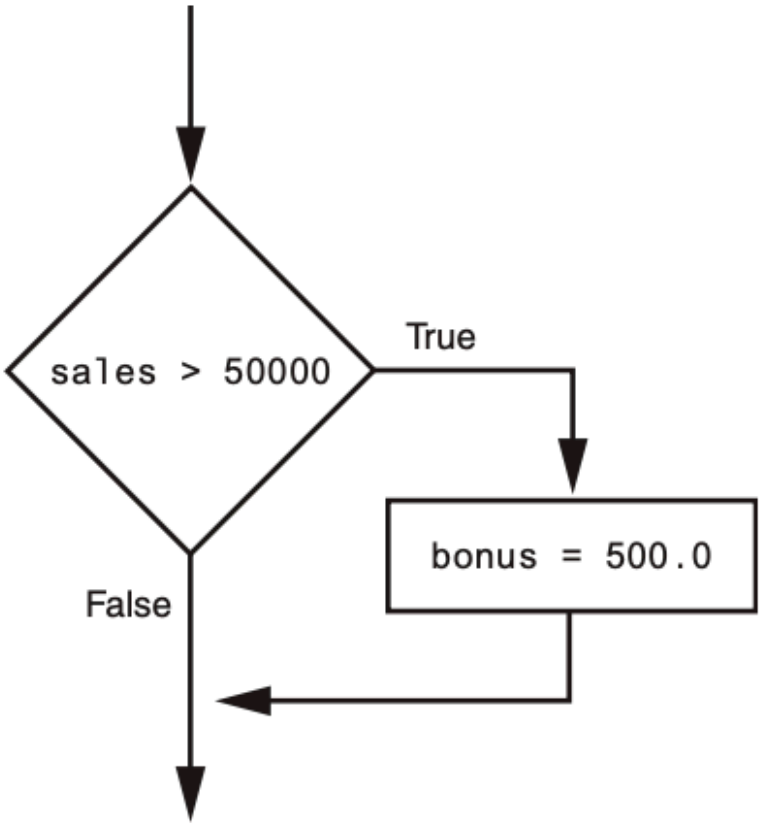


```
x = 1
y = 0
z = 1
x == y
x == z
y == z
```

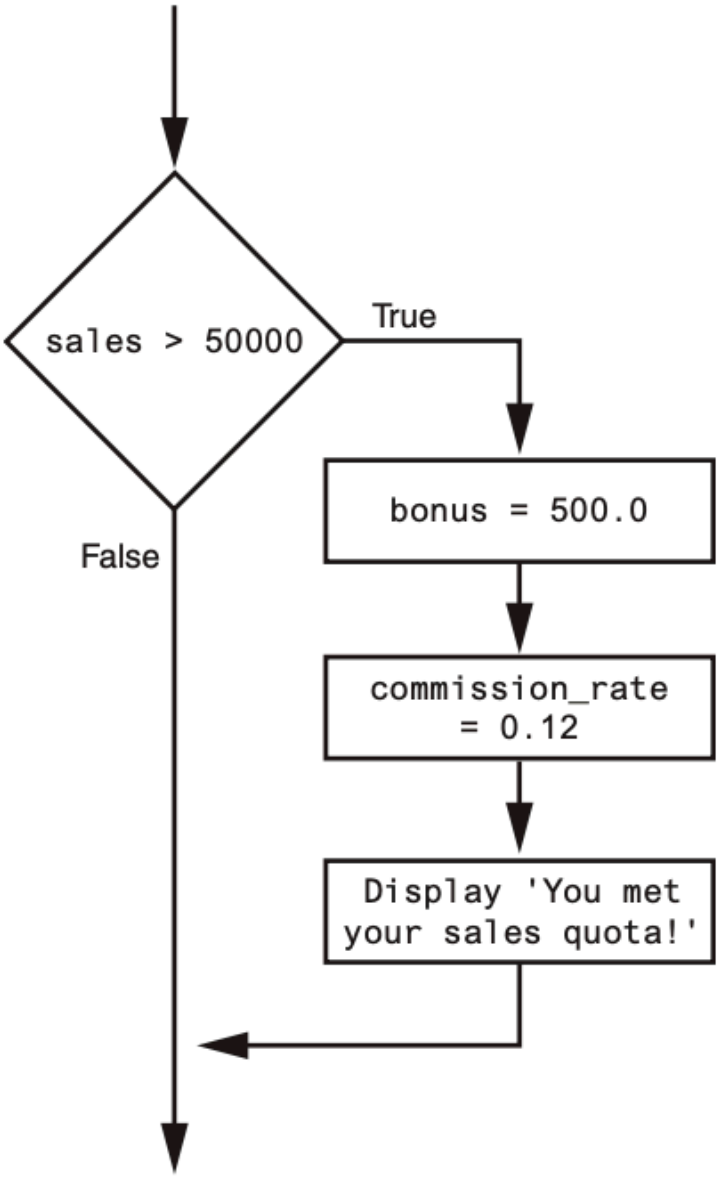
```
x = 1
y = 0
z = 1
x != y
x != z
```

Putting It All Together

- This statement uses the > operator to determine whether sales is greater than 50,000.
- If the expression sales > 50000 is true, the variable bonus is assigned 500.0.
- If the expression is false, however, the assignment statement is skipped.



The following example conditionally executes a block containing three statements



- The following code uses the == operator to determine whether two values are equal.
- The expression balance == 0 will be true if the balance variable is assigned 0.
- Otherwise, the expression will be false.

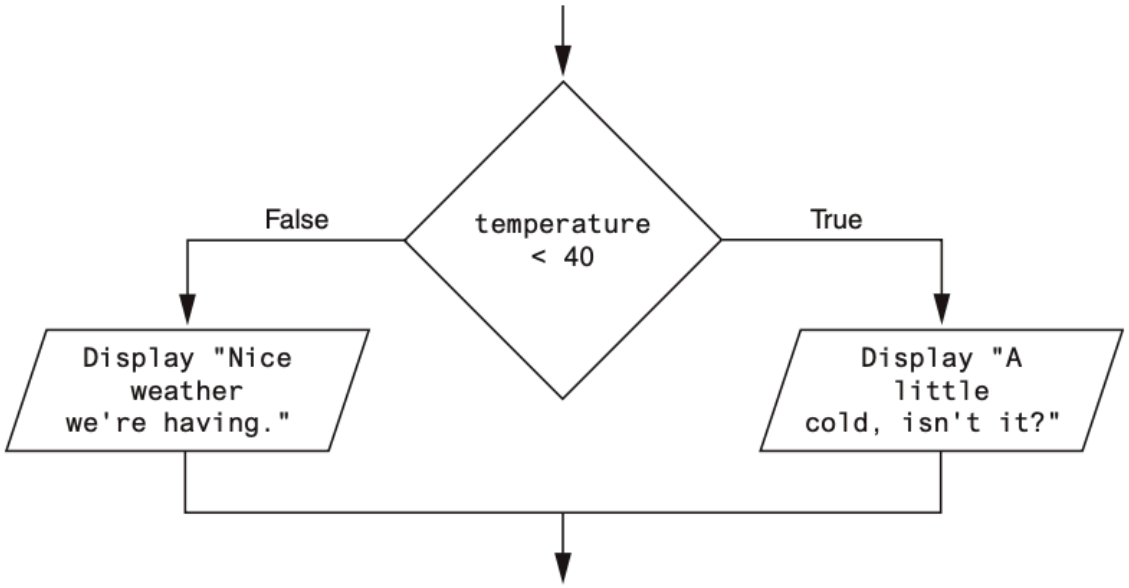
```
if balance == 0: # Statements appearing here will be executed only if balance is equal to 0.
```

- The following code uses the != operator to determine whether two values are not equal.
- The expression choice != 5 will be true if the choice variable does not reference the value 5.
- Otherwise, the expression will be false.


```
if balance != 5: # Statements appearing here will be executed only if balance is not equal to 5.
```

The if-else Statement

- The dual alternative decision structure, which has two possible paths of execution—one path is taken if a condition is true, and the other path is taken if the condition is false.

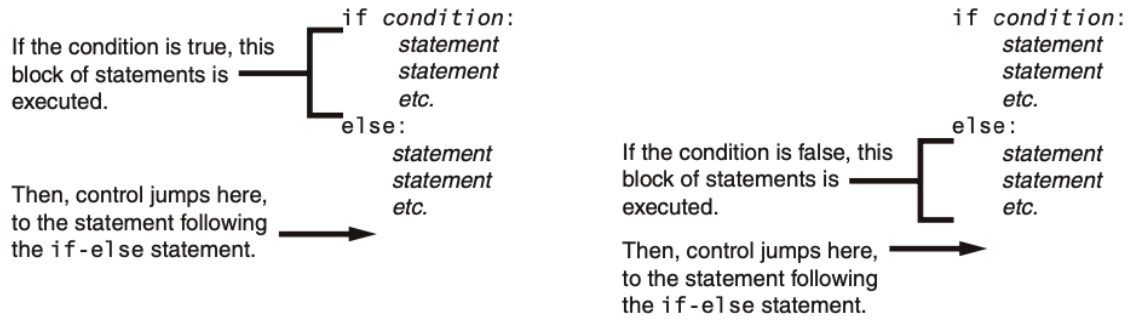


- In code, we write a dual alternative decision structure as an if-else statement.

```
if condition:
  statement
  statement
  etc.
else:
  statement
  statement
  etc.
```

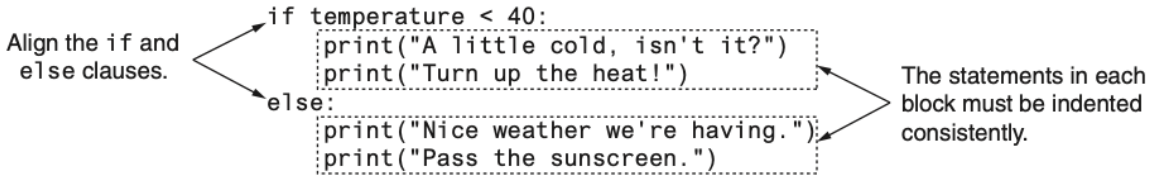
- When this statement executes, the condition is tested.
- If it is true,
  - the block of indented statements following the if clause is executed,
  - then control of the program jumps to the statement that follows the if-else statement.
- If the condition is false,
  - the block of indented statements following the else clause is executed,

- then control of the program jumps to the statement that follows the if-else statement.



Indentation in the if-else statement

- When you write an if-else statement, follow these guidelines for indentation:
  - Make sure the if clause and the else clause are aligned.
  - The if clause and the else clause are each followed by a block of statements.
  - Make sure the statements in the blocks are consistently indented.



Comparing Strings

Python allows you to compare strings. This allows you to create decision structures that test the value of a string

---

- You saw in the preceding examples how numbers can be compared in a decision structure.
- You can also compare strings. For example, look at the following code:


```
name1 = 'Mary'
name2 = 'Mark'
if name1 == name2:
  print('The names are the same.')
else:
  print('The names are NOT the same.')
```

- The == operator compares name1 and name2 to determine whether they are equal.
- Because the strings ‘Mary’ and ‘Mark’ are not equal, the else clause will display the message ‘The names are NOT the same.’

- Assume the month variable references a string.
- The following code uses the != operator to determine whether the value referenced by month is not equal to ‘October’:

```
if month != 'October':
  print('This is the wrong time for Octoberfest!')
```

Other String Comparisons

- In addition to determining whether strings are equal or not equal, you can also determine whether one string is greater than or less than another string.
- This is a useful capability because programmers commonly need to design programs that sort strings in some order.
- Computer store numeric codes that represent the characters.
- Chapter 1 mentioned that ASCII (the American Standard Code for Information Interchange) is a commonly used character coding system.
- You can see the set of ASCII codes in Appendix C, but here are some facts about it:

  The uppercase characters A through Z are represented by the numbers 65 through 90. The lowercase characters a through z are represented by the numbers 97 through 122. When the digits 0 through 9 are stored in memory as characters, they are represented by the numbers 48 through 57. (For example, the string ‘abc123’ would be stored in memory as the codes 97, 98, 99, 49, 50, and 51.)

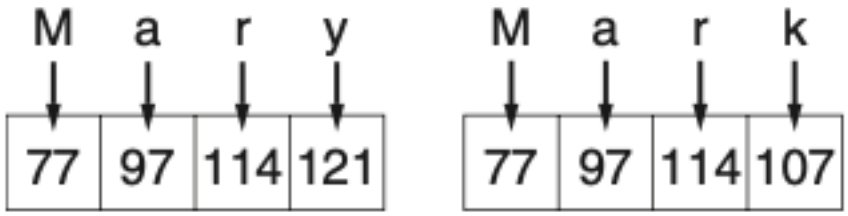
- A blank space is represented by the number 32.
- In addition to establishing a set of numeric codes to represent characters in memory, ASCII also establishes an order for characters.
- The character “A” comes before the character “B”, which comes before the character “C”, and so on.
- When a program compares characters, it actually compares the codes for the characters.


```
if 'a' < 'b':
  print('The letter a is less than the letter b.')
```

- This code determines whether the ASCII code for the character ‘a’ is less than the ASCII code for the character ‘b’.
- The expression ‘a’ < ‘b’ is true because the code for ‘a’ is less than the code for ‘b’.
- So, if this were part of an actual program it would display the message ‘The letter a is less than the letter b.’

```
name1 = 'Mary'
name2 = 'Mark'
```

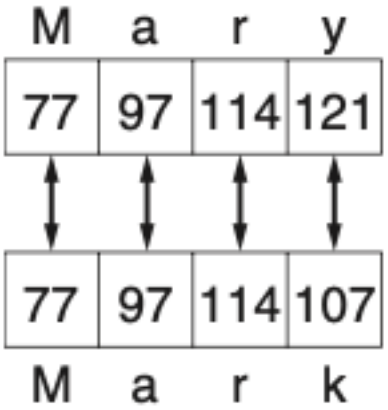
- The following shows how the individual characters in the strings ‘Mary’ and ‘Mark’ would actually be stored in memory, using ASCII codes.



- When you use relational operators to compare these strings, the strings are compared character-by-character.

```
name1 = 'Mary'
name2 = 'Mark'
if name1 > name2:
  print('Mary is greater than Mark')
else:
  print('Mary is not greater than Mark')
```

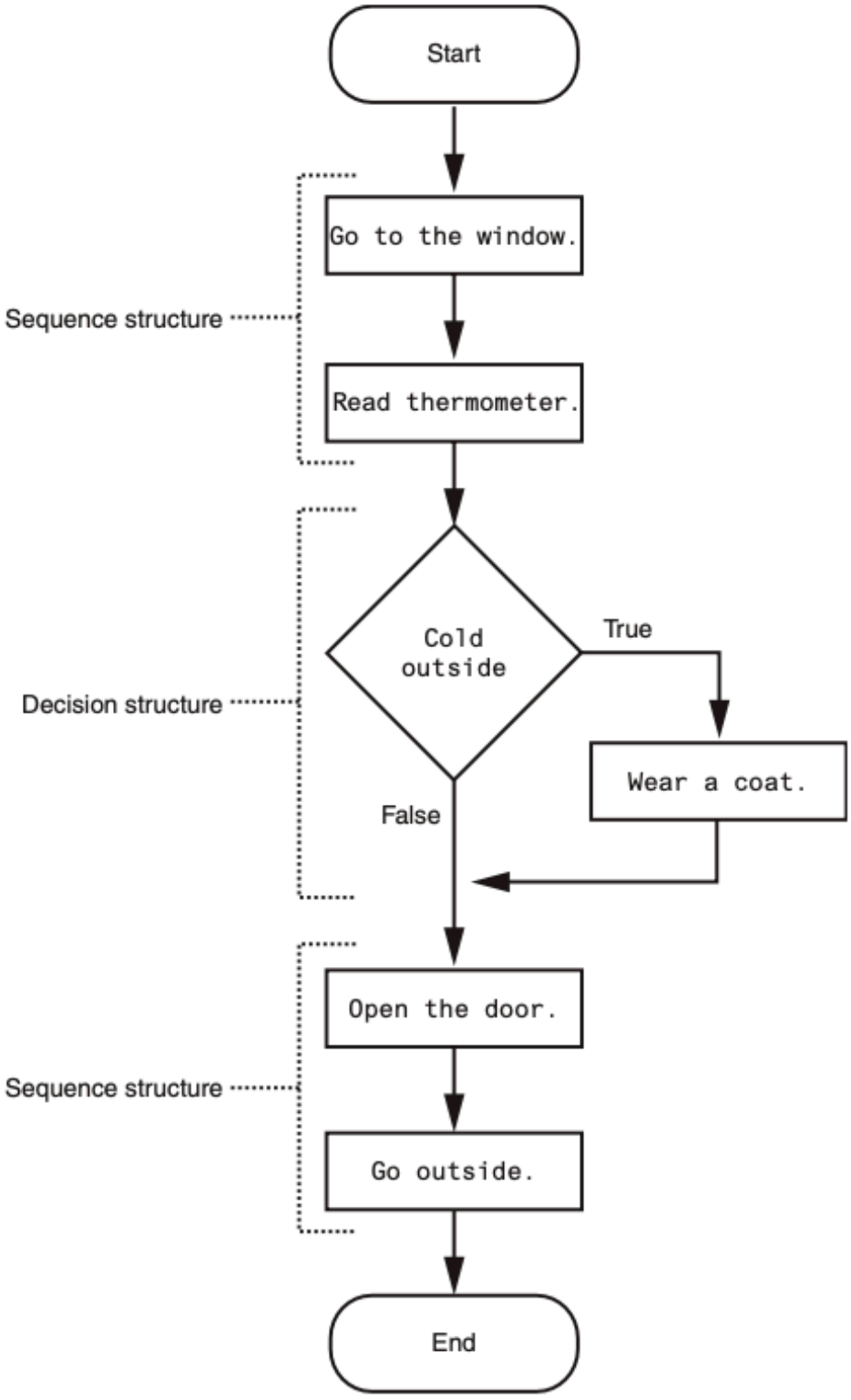
- The > operator compares each character in the strings ‘Mary’ and ‘Mark’, beginning with the first, or leftmost, characters.



- Here is how the comparison takes place:
  - The ‘M’ in ‘Mary’ is compared with the ‘M’ in ‘Mark’. Since these are the same, the next characters are compared.
  - The ‘a’ in ‘Mary’ is compared with the ‘a’ in ‘Mark’. Since these are the same, the next characters are compared.
  - The ‘r’ in ‘Mary’ is compared with the ‘r’ in ‘Mark’. Since these are the same, the next characters are compared.
  - The ‘y’ in ‘Mary’ is compared with the ‘k’ in ‘Mark’. Since these are not the same, the two strings are not equal. The character ‘y’ has a higher ASCII code (121) than ‘k’ (107), so it is determined that the string ‘Mary’ is greater than the string ‘Mark’.
- If one of the strings in a comparison is shorter than the other, only the corresponding characters will be compared.
- If the corresponding characters are identical, then the shorter string is considered less than the longer string.
  - For example, suppose the strings ‘High’ and ‘Hi’ were being compared. The string ‘Hi’ would be considered less than ‘High’ because it is shorter.

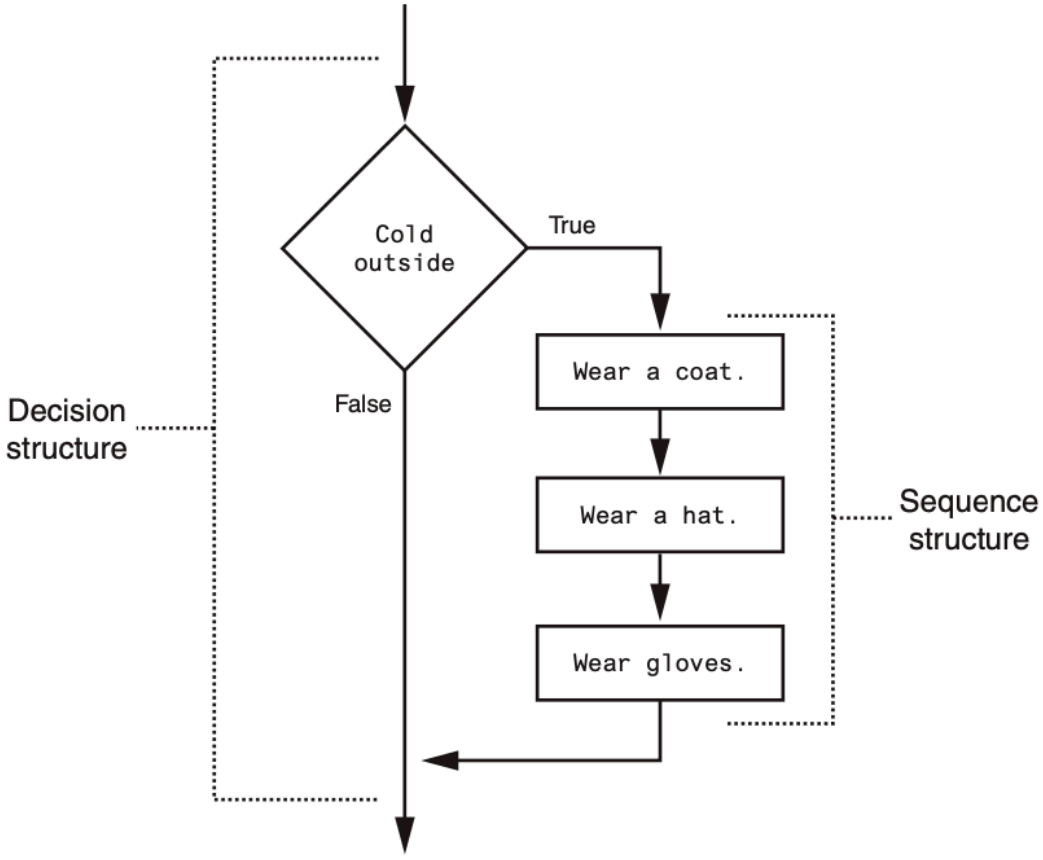
Nested Decision Structures and the if-elif-else Statement

Programs are usually designed as combinations of different control structures.

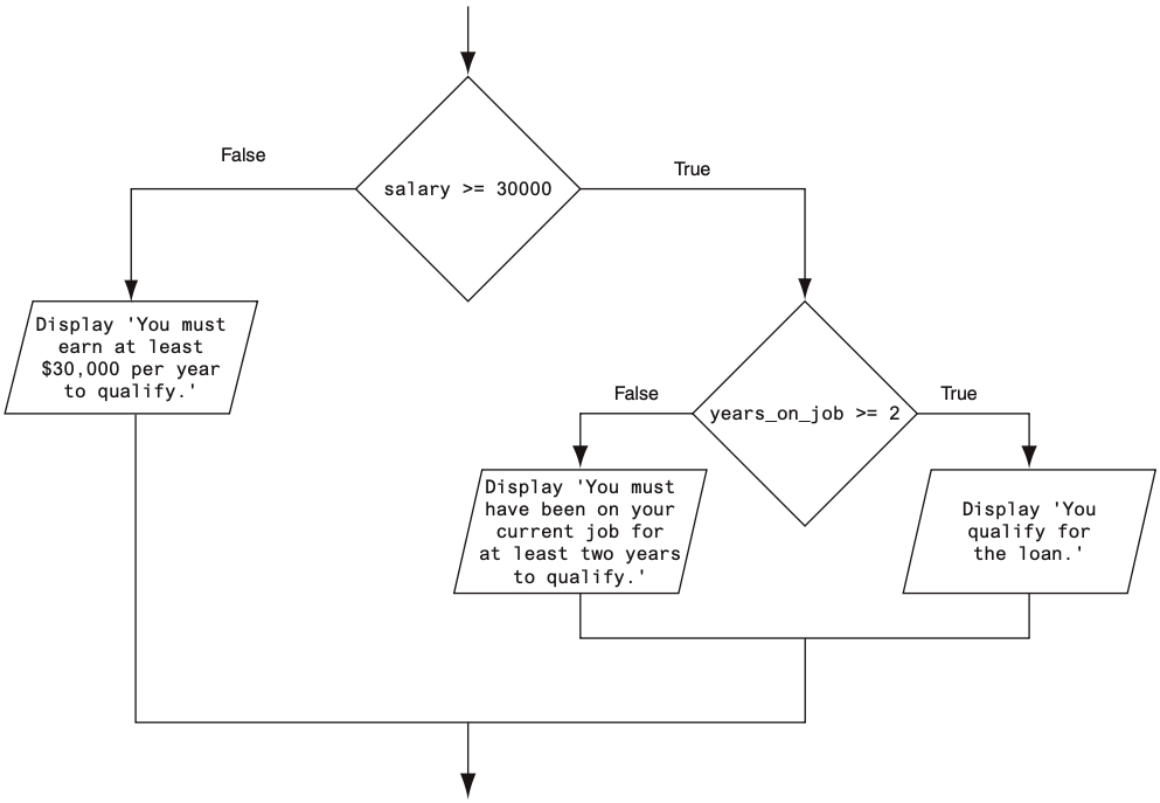


171

- The flowchart in the figure starts with a sequence structure.
- Assuming you have an outdoor thermometer in your window, the first step is Go to the window, and the next step is Read thermometer.
- A decision structure appears next, testing the condition Cold outside.
- If this is true, the action Wear a coat is performed. Another sequence structure appears next.
- The step Open the door is performed, followed by Go outside.
- Quite often, structures must be nested inside other structures.
  - For example, look at the partial flowchart in Figure 3-11.
  - It shows a decision structure with a sequence structure nested inside it.
  - The decision structure tests the condition Cold outside.
  - If that condition is true, the steps in the sequence structure are executed.



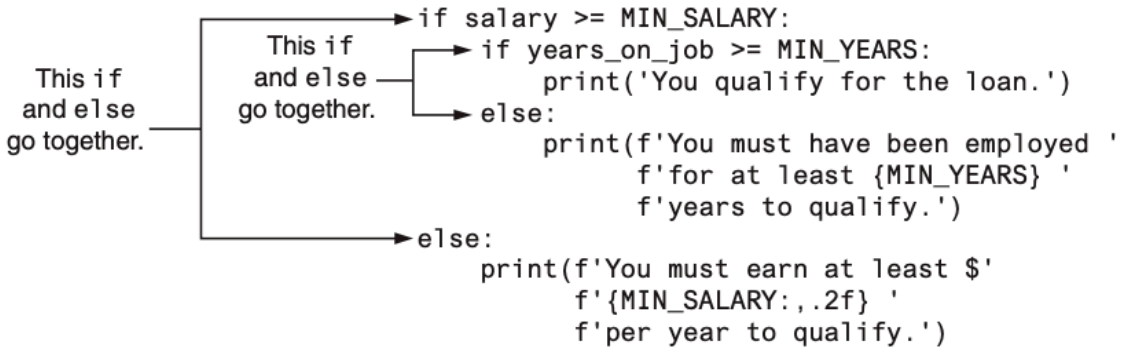
- You can also nest decision structures inside other decision structures.
- In fact, this is a common requirement in programs that need to test more than one condition.
- For example, consider a program that determines whether a bank customer qualifies for a loan.
  - To qualify, two conditions must exist:
    - (1) the customer must earn at least $30,000 per year, and
    - (2) the customer must have been employed for at least two years.



- If we follow the flow of execution, we see that the condition salary >= 30000 is tested.
- If this condition is false, there is no need to perform further tests; we know the customer does not qualify for the loan.
- If the condition is true, however, we need to test the second condition.
- This is done with a nested decision structure that tests the condition years_on_job >= 2.
- If this condition is true, then the customer qualifies for the loan.
- If this condition is false, then the customer does not qualify.

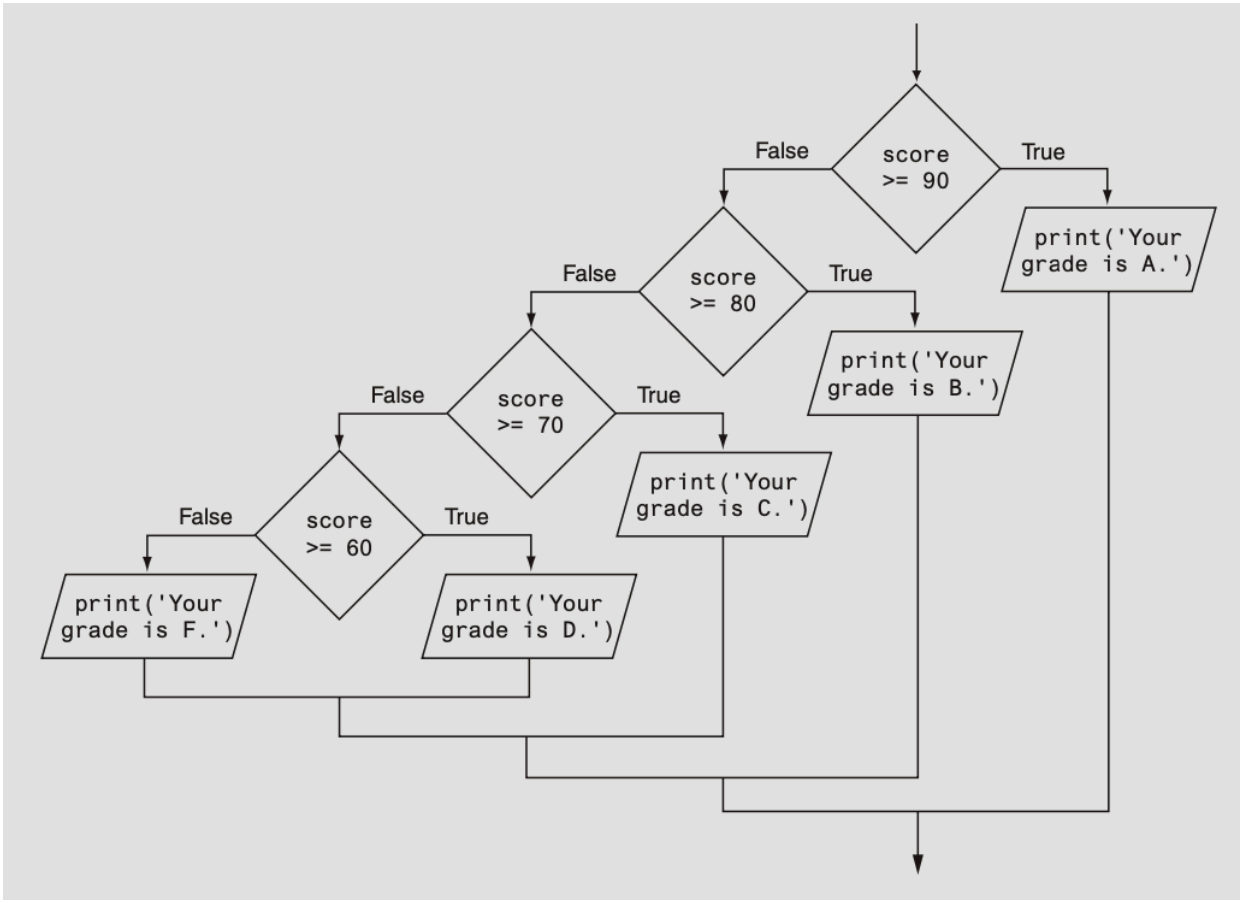
- Look at the if-else statement that begins in line 15.
- It tests the condition salary >= MIN_SALARY.
- If this condition is true, the if-else statement that begins in line 16 is executed.
- Otherwise the program jumps to the else clause in line 22 and executes the statement in lines 23 through 25.
- It’s important to use proper indentation in a nested decision structure.
- Not only is proper indentation required by the Python interpreter, but it also makes it easier for you, the human reader of your code, to see which actions are performed by each part of the structure.

- Follow these rules when writing nested if statements:
  - Make sure each else clause is aligned with its matching if clause.
  - Make sure the statements in each block are consistently indented.
  - The shaded parts of Figure 3-14 show the nested blocks in the decision structure.
  - Notice each statement in each block is indented the same amount.



Testing a series of conditions

- In the previous example, you saw how a program can use nested decision structures to test more than one condition.
- It is not uncommon for a program to have a series of conditions to test, then perform an action depending on which condition is true.
- One way to accomplish this is to have a decision structure with numerous other decision structures nested inside it.
- For example, consider the program presented in the following In the Spotlight section.



The if-elif-else Statement

- The logic of the nested decision structure is fairly complex.
- Python provides a special version of the decision structure known as the if-elif-else statement, which makes this type of logic simpler to write.
- Here is the general format of the if-elif-else statement:

```
if condition_1:
  statement
  statement
  etc.
elif condition_2:
  statement
  statement
  etc.
else:
  statement
```



```
    statement
etc.
```

- When the statement executes, condition_1 is tested.
- If condition_1 is true, the block of statements that immediately follow is executed, up to the elif clause.
  - The rest of the structure is ignored.
- If condition_1 is false, however, the program jumps to the very next elif clause and tests condition_2.
  - If it is true, the block of statements that immediately follow is executed, up to the next elif clause.
  - The rest of the structure is then ignored.
- This process continues until a condition is found to be true, or no more elif clauses are left.
- If no condition is true, the block of statements following the else clause is executed.

- Notice the alignment and indentation that is used with the if-elif-else statement: The if, elif, and else clauses are all aligned, and the conditionally executed blocks are indented.
- The if-elif-else statement is never required because its logic can be coded with nested if-else statements.
- However, a long series of nested if-else statements has two particular disadvantages when you are debugging code

---

- The code can grow complex and become difficult to understand.
- Because of the required indentation, a long series of nested if-else statements can become too long to be displayed on the computer screen without horizontal scrolling.
- Also, long statements tend to “wrap around” when printed on paper, making the code even more difficult to read.
- The logic of an if-elif-else statement is usually easier to follow than a long series of nested if-else statements.
- And, because all of the clauses are aligned in an if-elif-else statement, the lengths of the lines in the statement tend to be shorter.

Logical Operators

The logical and operator and the logical or operator allow you to connect multiple Boolean expressions to create a compound expression. The logical not operator reverses the truth of a Boolean expression.

Logical Operator

Operator	Meaning
and	The and operator connects two Boolean expressions into one compound expression. Both subexpressions must be true for the compound expression to be true.
or	The or operator connects two Boolean expressions into one compound expression. One or both subexpressions must be true for the compound expression to be true. It is only necessary for one of the subexpressions to be true, and it does not matter which.
not	The not operator is a unary operator, meaning it works with only one operand. The operand must be a Boolean expression. The not operator reverses the truth of its operand. If it is applied to an expression that is true, the operator returns false. If it is applied to an expression that is false, the operator returns true.

Compound Boolean expressions using logical operators

Expression	Meaning
x > y and a < b	Is x greater than y AND is a less than b?
x == y or x == z	Is x equal to y OR is x equal to z?
not (x > y)	Is the expression x > y NOT true?

The and Operator

- The and operator takes two Boolean expressions as operands and creates a compound Boolean expression that is true only when both subexpressions are true.

```
if temperature < 20 and minutes > 12:
  print('The temperature is in the danger zone.')
```

- In this statement, the two Boolean expressions temperature < 20 and minutes > 12 are combined into a compound expression.
- The print function will be called only if temperature is less than 20 and minutes is greater than 12.
- If either of the Boolean subexpressions is false, the compound expression is false and the message is not displayed.

- The truth table lists expressions showing all the possible combinations of true and false connected with the and operator.

Expression	Value and Expression
true and false	false
false and true	false
false and false	false
true and true	ture

The or Operator

- The or operator takes two Boolean expressions as operands and creates a compound Boolean expression that is true when either of the subexpressions is true.

```
if temperature < 20 or temperature > 100:
  print('The temperature is too extreme')
```

- The print function will be called only if temperature is less than 20 or temperature is greater than 100.
- If either subexpression is true, the compound expression is true.
- All it takes for an or expression to be true is for one side of the or operator to be true.
- It doesn't matter if the other side is false or true.

Expression	Value and Expression
true and false	true
false and true	true
false and false	false
true and true	ture

The not Operator

- The not operator is a unary operator that takes a Boolean expression as its operand and reverses its logical value.
- In other words, if the expression is true, the not operator returns false, and if the expression is false, the not operator returns true.

```
if not(temperature > 100):
  print('This is below the maximum temperature.')
```

- First, the expression (temperature > 100) is tested and a value of either true or false is the result.
- Then the not operator is applied to that value.
- If the expression (temperature > 100) is true, the not operator returns false.
- If the expression (temperature > 100) is false, the not operator returns true.
- The previous code is equivalent to asking: "Is the temperature not greater than 100?"

Expression	Value and Expression
not true	false
not false	true

Checking Numeric Ranges with Logical Operators

- Sometimes you will need to design an algorithm that determines whether a numeric value is within a specific range of values or outside a specific range of values.
- When determining whether a number is inside a range, it is best to use the and operator.
  - For example, the following if statement checks the value in x to determine whether it is in the range of 20 through 40:

```
if x >= 20 and x <= 40:
  print('The value is in the acceptable range.')
```

- The compound Boolean expression being tested by this statement will be true only when x is greater than or equal to 20 and less than or equal to 40.
- The value in x must be within the range of 20 through 40 for this compound expression to be true.
- When determining whether a number is outside a range, it is best to use the or operator.

```
if x < 20 or x > 40:
  print('The value is outside the acceptable range.')
```

- It is important not to get the logic of the logical operators confused when testing for a range of numbers.
- For example, the compound Boolean expression in the following code would never test true:
  - Obviously, x cannot be less than 20 and at the same time be greater than 40.

```
# This is an error!
if x < 20 and x > 40:
  print('The value is outside the acceptable range.')
```

Boolean Variables

A Boolean variable can reference one of two values: True or False. Boolean variables are commonly used as flags, which indicate whether specific conditions exist.

- In addition to these data types(float, int, str), Python also provides a bool data type.

- The bool data type allows you to create variables that may reference one of two possible values:
  - True or False.

```
hungry = True
sleepy = False
```

- Boolean variables are most commonly used as **flags**.
- A flag is a variable that signals when some condition exists in the program.
- When the flag variable is set to False, it indicates the condition does not exist.
- When the flag variable is set to True, it means the condition does exist.

```
if sales >= 50000.0:
  sales_quota_met = True
else:
  sales_quota_met = False
```

- As a result of this code, the sales_quota_met variable can be used as a flag to indicate whether the sales quota has been met.

```
if sales_quota_met:
  print('You have met your sales quota!')
```

- This code displays 'You have met your sales quota!' if the bool variable sales_quota_met is True.
- Notice we did not have to use the == operator to explicitly compare the sales_quota_met variable with the value True.

```
if sales_quota_met == True:
  print('You have met your sales quota!')
```

Turtle Graphics: Determining the State of the Turtle

The turtle graphics library provides numerous functions that you can use in decision structures to determine the state of the turtle and conditionally perform actions.

- We will discuss functions for determining
  - the turtle's location, the direction in which it is heading,
  - whether the pen is up or down,
  - the current drawing color, and more.

Determining the Turtle's Location

- You can use the turtle.xcor() and turtle.ycor() functions to get the turtle's current X and Y coordinates.
- The following code snippet uses an if statement to determine whether the turtle's X coordinate is greater than 249, or the turtle's Y coordinate is greater than 349.
- If so, the turtle is repositioned at (0, 0):

```
if turtle.xcor() > 249 or turtle.ycor() > 349:
  turtle.goto(0, 0)
```

Determining the Turtle's Heading

- The turtle.heading() function returns the turtle's heading. By default, the heading is returned in degrees. The following interactive session demonstrates this:

```
>>> turtle.heading()
0.0
>>>
```

- The following code snippet uses an if statement to determine whether the turtle's heading is between 90 degrees and 270 degrees. If so, the turtle's heading is set to 180 degrees:

```
if turtle.heading() >= 90 and turtle.heading() <= 270:
  turtle.setheading(180)
```

Determining Whether the Pen Is Down

- The turtle.isdown() function returns True if the turtle's pen is down, or False otherwise. The following interactive session demonstrates this:

```
>>> turtle.isdown()
True
>>>
```

- The following code snippet uses an if statement to determine whether the turtle's pen is down. If the pen is down, the code raises it:

```
if turtle.isdown():
  turtle.penup()
```

- To determine whether the pen is up, you use the not operator along with the turtle. isdown() function. The following code snippet demonstrates this:


```
if not(turtle.isdown()):
  turtle.pendown()
```

Determining Whether the Turtle Is Visible

- The turtle.isvisible() function returns True if the turtle is visible, or False other- wise. The following interactive session demonstrates this:

```
>>> turtle.isvisible()
True
>>>
```

- The following code snippet uses an if statement to determine whether the turtle is visible. If the turtle is visible, the code hides it:

```
if turtle.isvisible():
  turtle.hideturtle()
```

Determining the Current Colors

- When you execute the turtle.pencolor() function without passing it an argument, the function returns the pen's current drawing color as a string. The following interactive session demonstrates this:

```
>>> turtle.pencolor()
'black'
>>>
```

- The following code snippet uses an if statement to determine whether the pen's current color is red. If the color is red, the code changes it to blue:

```
if turtle.pencolor() == 'red':
  turtle.pencolor('blue')
```

- When you execute the turtle.fillcolor() function without passing it an argument, the function returns the current fill color as a string. The following interactive session demonstrates this:

```
>>> turtle.fillcolor()
'black'
>>>
```

- The following code snippet uses an if statement to determine whether the current fill color is blue. If the fill color is blue, the code changes it to white:


```
if turtle.fillcolor() == 'blue':
  turtle.fillcolor('white')
```

- When you execute the turtle.bgcolor() function without passing it an argument, the function returns the current background color of the turtle's graphics window as a string. The following interactive session demonstrates this:

```
>>> turtle.bgcolor()
'white'
>>>
```

- The following code snippet uses an if statement to determine whether the current background color is white. If the background color is white, the code changes it to gray:

```
if turtle.bgcolor() == 'white':
  turtle.fillcolor('gray')
```

Determining the Pen Size

- When you execute the turtle.pensize() function without passing it an argument, the function returns the pen's current size. The following interactive session demonstrates this:

```
>>> turtle.pensize()
1
>>>
```

- The following code snippet uses an if statement to determine whether the pen's current size is less than 3. If the pen size is less than 3, the code changes it to 3:

```
if turtle.pensize() < 3:
  turtle.pensize(3)
```

Determining the Turtle's Animation Speed

- When you execute the turtle.speed() function without passing it an argument, the function returns the turtle's current animation speed. The following interactive session demonstrates this:

```
>>> turtle.speed()
3
>>>
```

- The turtle's animation speed is a value in the range of 0 to 10.
- If the speed is 0, then animation is disabled, and the turtle makes all of its moves instantly.

- If the speed is in the range of 1 through 10, then 1 is the slowest speed and 10 is the fastest speed.
- The following code snippet determines whether the turtle's speed is greater than 0. If it is, then the speed is set to 0:

```
if turtle.speed() > 0:
  turtle.speed(0)
```

- It uses an if-elif-else statement to determine the turtle's speed, then set the pen color.
- If the speed is 0, the pen color is set to red.
- Otherwise, if the speed is greater than 5, the pen color is set to blue.
- Otherwise, the pen color is set to green:

```
if turtle.speed() == 0:
  turtle.pencolor('red')
elif turtle.speed() > 5:
  turtle.pencolor('blue')
else:
  turtle.pencolor('green')
```

Reference

Gaddis, T. (2022). *Starting out with python 5th*. Pearson.

